

MI AI API

Version 3.54

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.50	Initial release	12/04/2020
3.51	Audio new architecture	07/19/2021
	Added procfs introduction	08/25/2021
3.52	Added MI_AI_GetAttr	09/08/2021
	Modified version: Audio 3.0 Version starts from 3.50(3.0 to 3.49 are Audio 2.0 versions)	12/29/2021
3.53	Added Mochi info	02/17/2022
3.54	Added Maruko info	03/29/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. Overview.....	1
1.1. Module Description	1
1.2. Audio Codec Block Diagram	2
1.2.1 Muffin series	2
1.2.2 Mochi series	3
1.2.3 Maruko series	4
1.3. Audio Codec Device Description	4
1.4. Audio Codec Difference Description	5
1.4.1 Muffin series	5
1.4.2 Mochi series	6
1.4.3 Maruko series	6
1.5. Keyword	6
2. API reference.....	11
2.1. MI_AI_Open	11
2.2. MI_AI_OpenWithCfgFile	14
2.3. MI_AI_Close	16
2.4. MI_AI_AttachIf	17
2.5. MI_AI_EnableChnGroup	18
2.6. MI_AI_DisableChnGroup	19
2.7. MI_AI_Read	19
2.8. MI_AI_ReleaseData	21
2.9. MI_AI_SetGain	21
2.10. MI_AI_GetGain	22
2.11. MI_AI_SetMute	23
2.12. MI_AI_GetMute	24
2.13. MI_AI_SetIfGain	25
2.14. MI_AI_GetIfGain	26
2.15. MI_AI_SetIfMute	27
2.16. MI_AI_GetIfMute	28
2.17. MI_AI_SetI2SConfig	29
2.18. MI_AI_GetI2SConfig	29
2.19. MI_AI_DupChnGroup	30
2.20. MI_AI_InitDev	31
2.21. MI_AI_DeInitDev	31
2.22. MI_AI_GetAttr	32
3. AI Data type.....	34
3.1. MI_AUDIO_DEV	34
3.2. MI_AUDIO_Format_e	35
3.3. MI_AUDIO_SoundMode_e	35
3.4. MI_AUDIO_SampleRate_e	36
3.5. MI_AI_Attr_t	37

3.6.	MI_AI_If_e	38	
3.7.	MI_AI_MAX_CHN_NUM	40	
3.8.	MI_AI_Data_t	40	
3.9.	MI_AUDIO_I2sMode_e	41	
3.10.	MI_AUDIO_I2sBitWidth_e	41	
3.11.	MI_AUDIO_I2sFormat_e	42	
3.12.	MI_AUDIO_I2sMclk_e	42	
3.13.	MI_AUDIO_I2sConfig_t	43	
3.14.	MI_AI_InitParam_t	44	
4.	Error Code		45
5.	PROCFS INTRODUCTION		46
5.1.	cat	46	
5.2.	echo	48	

1. OVERVIEW

1.1. Module Description

Audio Input (AI) is mainly used to configure and enable audio input devices, read Audio data, and adjust volume.

1.2. Audio Codec Block Diagram

1.2.1 Muffin series

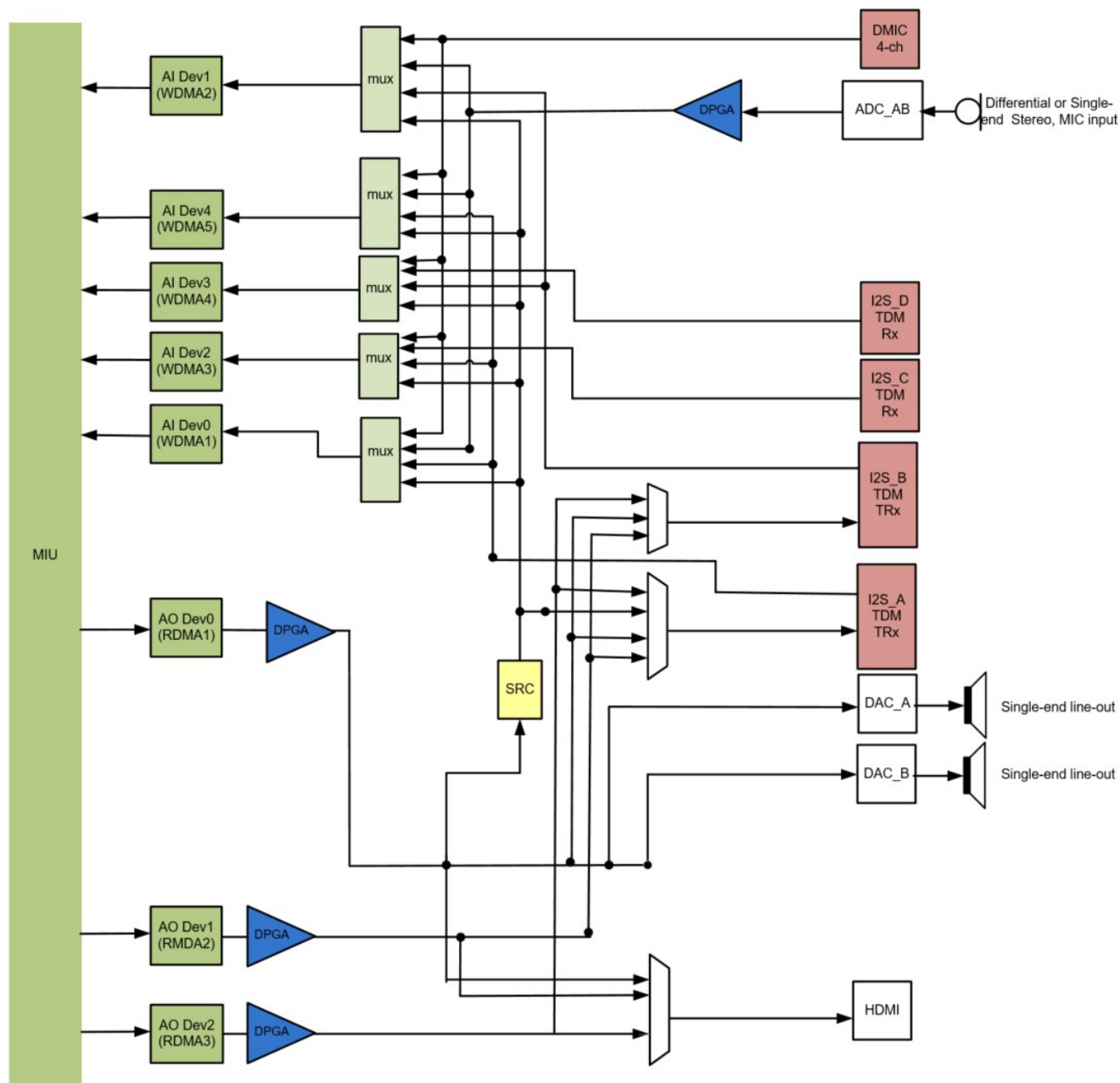


Figure 1-1: Muffin series chips

The audio codec of Muffin series chips has the following resources: WDMA * 5, RDMA * 3, DMIC interface (support 4Chn DMIC signal) * 1, ADC (support 2Chn Amic/Linein) * 2, I2S-TDM RX * 4, I2S -TDM TX * 2, DAC (support 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1.

1.2.2 Mochi series

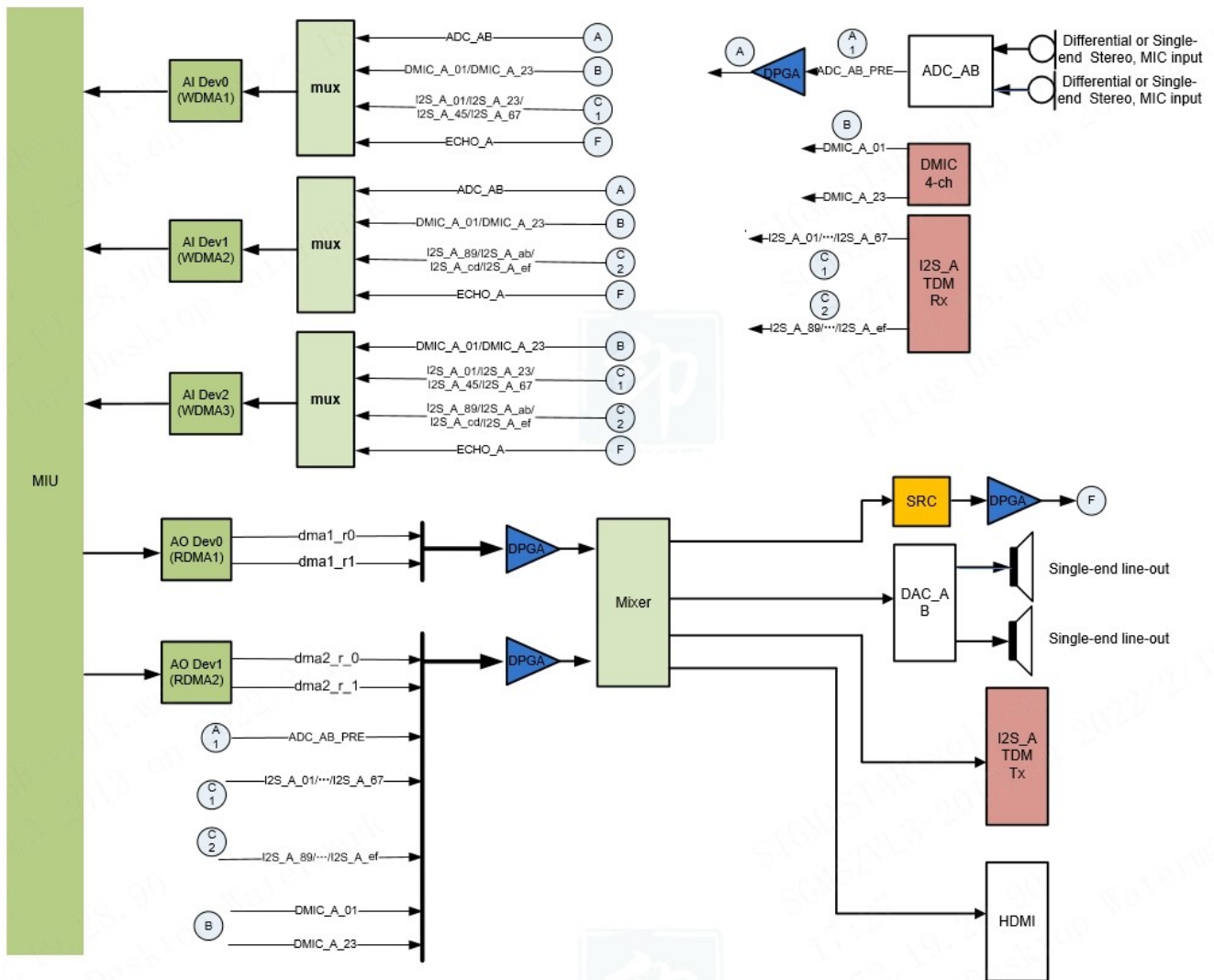


Figure 1-2: Mochi series chips

The audio codec of Mochi series chips has the following resources: WDMA * 3, RDMA * 2, DMIC interface (support 4Chn DMIC signal) * 1, ADC (support 2Chn Amic/Linein) * 2, I2S-TDM RX(16 slot) * 1, I2S-TDM TX(16 slot, but there are only two channels of valid data) * 1, DAC (support 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1.

1.2.3 Maruko series

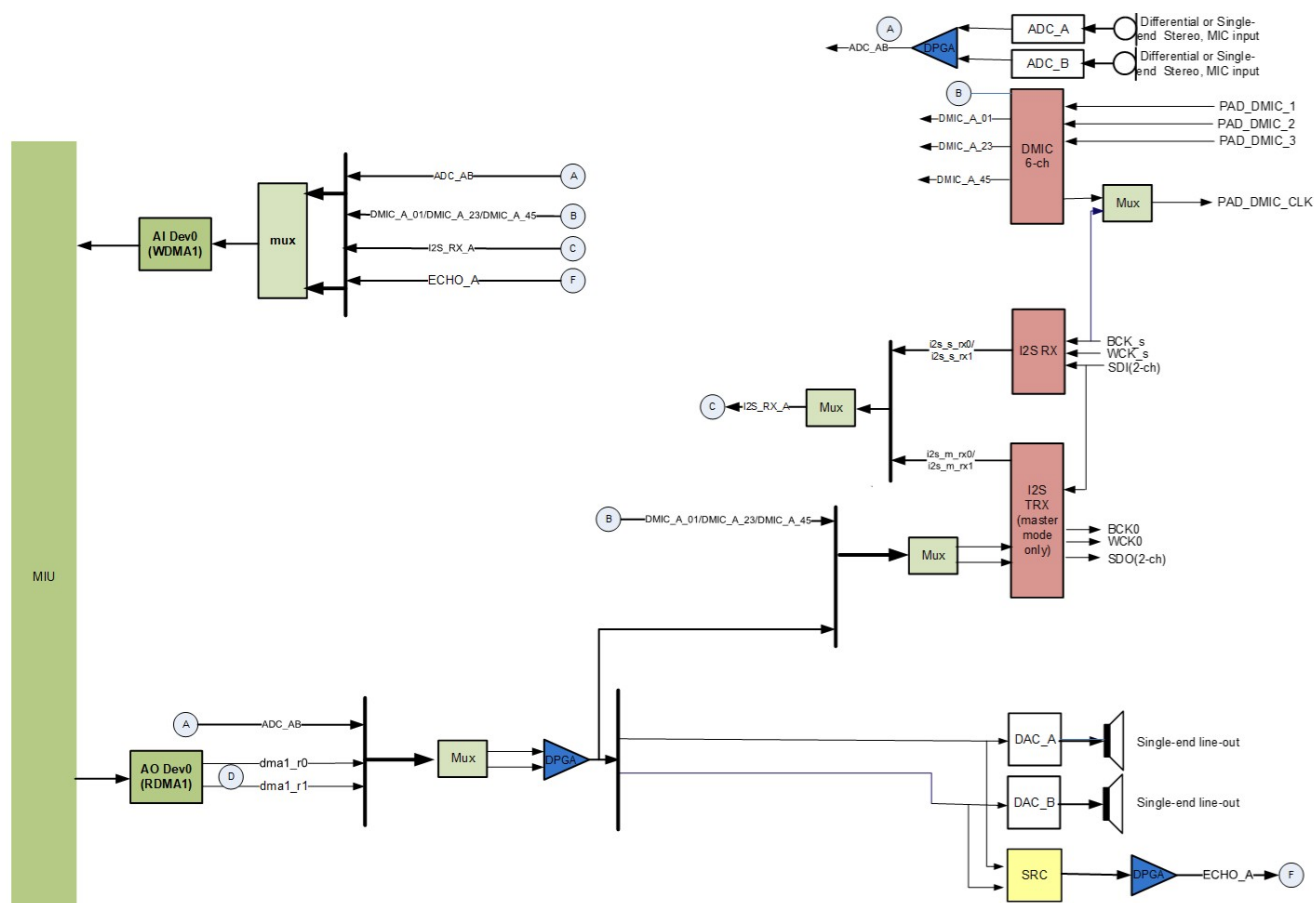


Figure 1-3: Maruko series chips

The audio codec of Maruko series chips has the following resources: WDMA * 1, RDMA * 1, DMIC interface (support 6Chn DMIC signal) * 1, ADC (support 2Chn Amic/Linein) * 2, I2S RX(2 slot) * 2, I2S TX(2 slot) * 1, DAC (support 2Chn Lineout) * 2, SRC * 1.

1.3. Audio Codec Device Description

- DMA
Direct Memory Access, DMA transfer copies data from one address space to another address space, and provides high-speed data transfer between peripherals and memory or between memory and memory. When the CPU initializes this transfer action, the transfer action itself is realized and completed by the DMA controller. The DMA transmission method does not require the CPU to directly control the transmission, and there is no interrupt processing method to retain the scene and restore the scene process. The hardware opens a channel for direct data transmission for RAM and IO devices, which greatly improves the efficiency of the CPU.
- WDMA
Direct Memory Access Writer.
- RDMA
Direct Memory Access Reader.

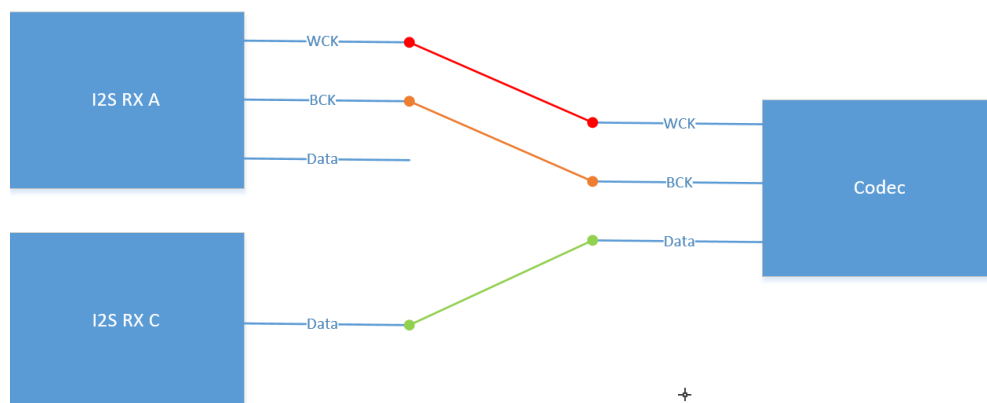
- **MUX**
MUX is the multiplexer data selector. In the process of multiplexing data transmission, any circuit that can be multiplexed out according to needs. The Mux in front of WDMA selects multiple data sources for WDMA, which can support the selection of 1/2/4 data sources (the data sources can be the same or different). Each data source has two channels, that is, the selection of 2/4/8 channels data is written to DRAM by WDMA, which acts as a multiplexer. However, the Mux, which is close to the output peripheral interface (such as I2S TX/HDMI/DAC, etc.), realizes the function of selecting one.
- **DPGA**
Digital Programmable Gain Amplifier, is a very versatile amplifier, and its amplification factor can be controlled by a program as needed.
- **DMIC**
Digital Microphone Interface, audio codec only provides DMIC interface, not a complete DMIC. The DMIC interface provides the clock signal required for DMIC work, and receives the PDM signal from the DMIC.
- **ADC**
Analog Digital Conversion, the electronic component that convert analog signals into digital signals.
- **I2S**
Inter-IC Sound integrated circuit built-in audio bus, is a bus standard developed by Philips for audio data transmission between digital audio devices. Sigmastar's I2S bus only supports the standard I2S data format and the left-justified I2S data format. At the same time, it also supports TDM (Time-Division Multiplexing) technology, which interleaves different signals in different time periods and transmits them along the same channel, which can support 4/8/16 (Mochi series chips support 16 slots) channel data transmission.
- **DAC**
Digital Analog Conversion, the electronic component that convert digital signals into analog signals.
- **SRC**
Sample Rate Convert.
- **HDMITX**
High Definition Multimedia Interface TX, is a fully digital video and sound transmission interface, which can transmit uncompressed audio and video signals.

1.4. Audio Codec Difference Description

1.4.1 Muffin series

Muffin series chips have I2S RX * 4, but I2S RX C and I2S RX D have two modes, one is called share mode and the other is called slave mode. In share mode, I2S RX C and I2S RX A share I2S Clock (Wck and Bck), and I2S RX D and I2S RX B share I2S Clock (Wck and Bck). When both I2S RX A and I2S RX C need to be used at the

same time The I2S parameters of the I2S must be completely consistent. The Codec that is connected to the I2S RX C needs to connect the I2S Clock pins (Wck and Bck) to the I2S Clock (Wck and Bck) of the I2S RX A, and the Data pin to the Data of the I2S RX C. Then, the hardware solution is shown in the figure below. Slave mode, I2S RX C and I2S RX D have independent clocks, but they can only be used as slaves. These two modes can be set by `i2s-rx-mode` under the `dtb` sound node, 0 is slave mode, and 1 is share mode.



1.4.2 Mochi series

The Mochi series chip I2S supports a maximum of 16 slots, and the sampling rate is newly added to support 96K/192K. Note: In the usage scenario of I2S, the maximum I2S BCK cannot exceed 30MHz (eg: $BCK = 16\text{bit} * 16\text{slots} * 192\text{K} = 49.152\text{M}$ / $BCK = 32\text{bit} * 16\text{slots} * 192\text{K} = 98.304\text{M}$, the usage of $BCK > 30\text{MHz}$ is not supported). MUX currently supports up to 8 channels, so if you need to use I2S RX 16slot, you need to attach 16-channel data to two DMAs.

1.4.3 Maruko series

Maruko series chips, although there are two sets of I2S RX, but two sets of I2S RX cannot be used at the same time. Here are two sets of I2S RX usage scenarios:

I2S RX:

- External codec transmits non-I2S signals, which need to be adjusted to Wck and Bck, such as receiving PCM signals.
- Need to synchronize with Dmic Share clock to achieve Dmic and I2S RX.
- External codec works in master mode.

I2S TRX:

- Need to use I2S TX.

The I2S RX is selected by using the `i2s-pcm` under the `dtb` sound node. 1 is I2S RX and 0 is I2S TRX. In addition, whether I2S RX and Dmic share clock are configured through `dmic-bck-share`. 1 is the share clock, and 0 is the independent clock.

1.5. Keyword

- Device (Audio input device)
AI Device refers to WDMA of audio codec. Device is an abstraction of DMA in audio codec by `audio3.0`. WDMA in audio codec and AI Device have a one-to-one correspondence. For example, AI Device0 corresponds to WDMA1 in audio codec, AI Device1 corresponds to WDMA2 in audio codec, and so on. The data streams of `audio3.0` are all connected in series with DMA as the center.

- Channel Group (Audio input channel group)

AI Channel Group refers to a collection of multiple physical channels. The number of physical channels in the set is determined by the Sound Mode of the AI Device. The meaning of Channel Group is to ensure the synchronization of related multiple physical channel data. For example, to make a 4Mic beamforming algorithm, the 4 physical channels must ensure data synchronization. The solution is to operate the 4 physical channels as a whole. This is the meaning of Channel Group. How many Channel Groups the AI Device shares is determined by the number of physical channels connected to the AI Device and the Sound Mode. Channel Group Number = (the number of physical channels connected to the Device - the number of physical channels connected to the Device of Echo) / Sound Mode.

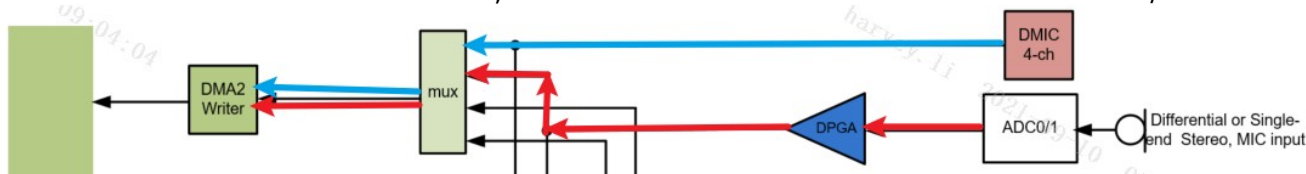
- Interface (Audio input peripherals)

AI Interface is an abstraction of audio input peripheral interfaces in audio codec, such as Amic/Dmic/I2S/HDMI interfaces.

- Attach

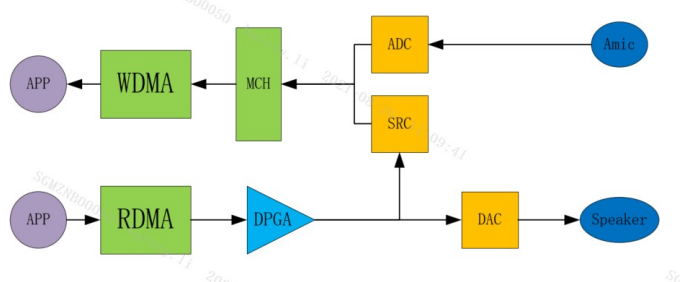
AI Attach refers to mounting Interface to the MUX corresponding to WDMA, that is, associating Device with Interface. For AI Device, Attach is to set the MUX corresponding to WDMA, select which Interface data can pass through the MUX, and WDMA will do the subsequent data transfer. AI Device does not support dynamic Attach Interface.

The following figure is the result of attaching Dmic and ADC to WDMA2. MUX selects Dmic and ADC0/1 as two data sources and connects to WDMA2, so that WDMA2 can receive data from Dmic and ADC0/1.



- Echo

AI Echo refers to the AEC reference data provided by the audio codec. It can be seen from the audio codec block diagram that the input of SRC is the signal output by RDMA and amplified by DPGA, and the output of SRC is the signal after re-sampling the input, which can be sent to WDMA through Multi Channel as the echo reference data of the AEC algorithm. For AI Device, Echo represents the output signal of SRC in the audio codec block diagram. For AO Device, Echo means to connect the output of AO Device to the input of SRC. The following figure shows the data flow of AI Device and AO Device using Echo at the same time with a simple block diagram. The application can obtain the aligned AEC far end and near end data.



- Sound Mode

AI Sound Mode refers to how many physical channels a channel group consists of, and how many Channel Groups a Device has depends on the number of physical channels on the attach and the Sound Mode.

Channel Group Number = (Physical Channels on Attach - Echo Channels on Attach) / Sound Mode.

- Gain

AI Gain is divided into two categories in the audio3.0 architecture. One is DPGA Gain associated with Device, that is, DPGA in the audio codec block diagram, and the other is Gain unique to Interface. At present, the Interface with Gain has ADC and Dmic interface. For AI, if the Interface itself does not have an independent Interface Gain and is not connected to Dpga, then Gain cannot be set.

- Format

What data format is used to represent an audio sample. Currently, only S16_LE format (PCM Linear 16bit (Little Endian)) is supported.

- Sample Rate

The frequency the recording device samples the analog signal per unit time. The higher the sampling frequency, the more realistic and natural the waveform of the mechanical wave.

- Period Size

For AI Device, Period Size represents the amount of data contained in each AI buffer (number of samples).

- Interleave

For AI Device, whether Interleave is turned on or not determines how MI_AI stores the data of the Channel Group. If Interleave is turned on, the data of each physical channel in the Channel Group is stored interleaved by sample. If Interleave is closed, the data of each physical channel in the Channel Group is stored separately.

- I2S parameter

1. I2S Mode

I2S Mode determines the working mode of I2S, whether it is standard I2S mode or Tdm I2S mode (2Channel or multi-channel), Master or Slave (Master provides synchronous clock, Slave receives synchronous clock). Generally speaking, there are no restrictions on the working mode, as long as it can match the external Codec clock.

2. I2S BitWidth

The bit width of I2S sending and receiving data, currently support 16/32bit, but the hardware can only process 16bit, which means that when the bit width is 32bit, the lower 16bit is invalid data.

3. I2S Format

I2S Format is the alignment of I2S. Currently, only I2S Philips and Left-justified alignment are supported. The following figure shows the waveforms of these two formats.

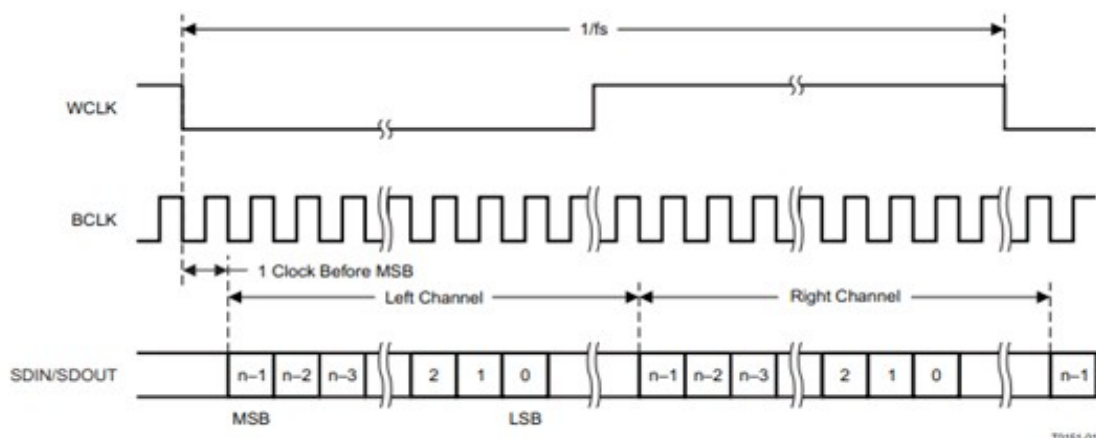


Figure 20. I²S Serial Data Bus Mode Operation

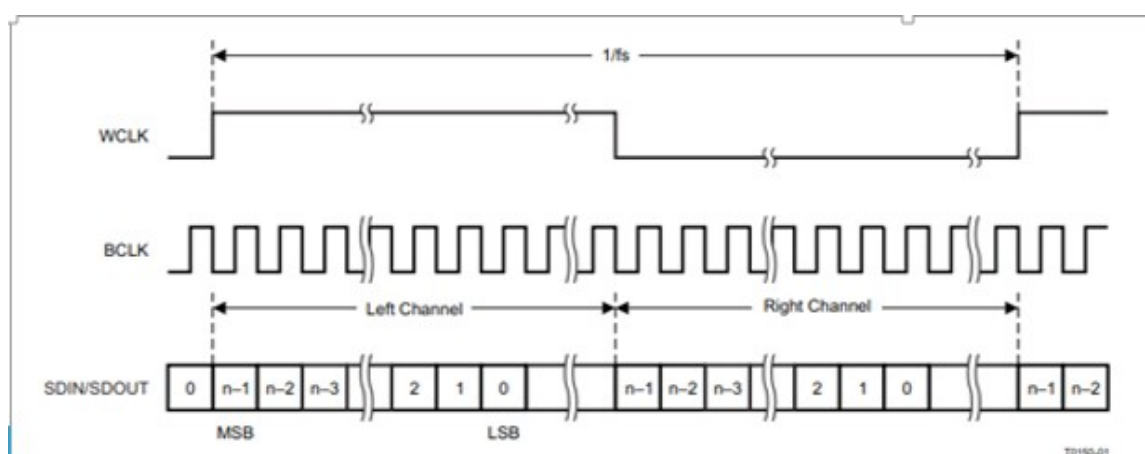


Figure 19. Left-Justified Serial Data Bus Mode Operation

I2S Philips alignment format, the first data bit of the sample data appears after the first BCLK (serial clock) of the WCLK (left and right channel switching clock) transition. In the left-aligned format, the first data bit of the sample data appears in the first BCLK (serial clock) of the WCLK (left and right channel switching clock) transition, and the polarity of WCLK is opposite to the alignment format of I2S Philips.

4. I2S Sample Rate
The sample frequency of I2S transmission and reception.
5. Mclk
Mclk, called the master clock, also called the system clock (System Clock), is generally 256 or 384 times the sample rate. The function is to enable better synchronization between systems, but it is not necessary. Currently only 12.288M, 16.384M, 18.432M, 24.576M, 24M, 48M, etc. are supported.
6. bSyncClock/4-Wire/6-Wire Mode
There are two wiring methods for SigmaStar's I2S. One is the 4-Wire mode, including RX_WCK, RX_BCK, RX_SDI, TX_SDO four wires. In this mode, TX does not have an independent clock, and all

clocks are provided by RX. Therefore, in this mode, TX needs to rely on RX to use. TX cannot be used alone, and the parameters of I2S TX must be consistent with I2S RX. The other is 6-Wire mode, including RX_WCK, RX_BCK, RX_SDI, TX_WCK, TX_BCK, TX_SDO six wires. In this mode, RX and TX are independent and not related. The choice of 4-Wire/6-Wire Mode needs to be decided according to specific scenarios.

If bSyncClock in the MI API I2S parameter is TRUE, 4-Wire Mode is used, and FALSE is 6-Wire Mode. The RX and TX belonging to the same group of I2S cannot be set to 4-Wire Mode on one side and 6-Wire Mode on the other side.

7. Slot

Slot represents the number of channels transmitted by I2S. Currently, 2 slots are supported in I2S mode, and 4/8 slots(Mochi series chips support 16 slots) are supported in Tdm mode.

2. API REFERENCE

Audio Input (AI) mainly implements functions such as configuring and enabling audio input devices and acquiring audio frame data.

API Name	Features
MI_AI_Open	Enable audio input device
MI_AI_OpenWithCfgFile	Enable the audio input device and initialize it according to the config file
MI_AI_Close	Close AI devices
MI_AI_AttachIf	Mount peripherals to AI device
MI_AI_EnableChnGroup	Enable AI channel group
MI_AI_DisableChnGroup	Disable AI channel group
MI_AI_Read	Read audio data
MI_AI_ReleaseData	Release audio data
MI_AI_SetGain	Set the volume of the AI channel group
MI_AI_GetGain	Get the volume of the AI channel group
MI_AI_SetMute	Set the mute parameters of the AI channel group
MI_AI_GetMute	Get the mute parameters of the AI channel group
MI_AI_SetIfGain	Set AI peripheral volume
MI_AI_GetIfGain	Get AI peripheral volume
MI_AI_SetIfMute	Set AI peripheral mute parameters
MI_AI_GetIfMute	Get AI peripheral mute parameters
MI_AI_SetI2SConfig	Set I2S RX config info
MI_AI_GetI2SConfig	Get I2S RX config info
MI_AI_DupChnGroup	Sync status of AI channel group
MI_AI_InitDev	Initialize the AI device
MI_AI_DeInitDev	De-Initialize the AI device
MI_AI_GetAttr	Get AI device attributes

2.1. MI_AI_Open

➤ Features

Enable audio input device.

➤ Syntax

MI_S32 [MI_AI_Open](#) ([MI_AUDIO_DEV](#) AiDevId, const [MI_AI_Attr_t](#) *pstAttr);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
pstAttr	Device property pointer	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: `mi_ai.h`
- Library: `libmi_ai.a/libmi_ai.so`

➤ Note

Audio input device attributes include data format, sound mode, sampling rate, audio data samples, and interleaving mode.

- Audio data format ([MI_AUDIO_Format_e](#))
The data format of the sampled samples. Only S16_LE is supported.
- Audio sound mode ([MI_AUDIO_SoundMode_e](#))
The physical channels included in the channel group.
MONO: 1 channel group contains 1 physical channel;
STEREO: 1 channel group contains 2 physical channels;
4CH: 1 channel group contains 4 physical channels;
6CH: 1 channel group contains 6 physical channels;
8CH: 1 channel group contains 8 physical channels.
- Audio sampling rate ([MI_AUDIO_SampleRate_e](#))
The number of samples in one second. The higher the sampling rate, the smaller the distortion, but the amount of data processed also increases.
- Audio data samples(u32PeriodSize)
The number of samples contained in the audio data read each time. When the audio sampling rate is high, it is recommended to increase u32PeriodSize accordingly. If the collected sound is not continuous, please increase u32PeriodSize and buffer size.
- Interlaced mode (bInterleaved)
Whether the interleaving mode is enabled determines the data arrangement of each physical channel in the channel group. Take for example when the sound mode is 4CH, the following is described with Sx instead of the xth sample. When the interleaving mode is enabled, the data of the 4 physical channels are interleaved, such as Chn0S0 Chn1S0 Chn2S0 Chn3S0 Chn0S1 Chn1S1 Chn2S1 Chn3S1.....;
When the interleaving mode is disabled, the data of the 4 physical channels are arranged in the order of the channels. As follows:
Chn0S0 Chn0S1 Chn0S2 Chn0S3 Chn0Sn
Chn1S0 Chn1S1 Chn1S2 Chn1S3 Chn1Sn

Chn2S0 Chn2S1 Chn2S2 Chn2S3 Chn2Sn
Chn3S0 Chn3S1 Chn3S2 Chn3S3 Chn3Sn

➤ Example

The simple example is as follow:

```
1. MI_AI_Attr_t stAiSetAttr = {0};
2. MI_AUDIO_DEV AiDevId = 0;
3. stAiSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
4. stAiSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
5. stAiSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAiSetAttr.u32PeriodSize = 1024;
7. stAiSetAttr.bInterleaved = TRUE;
8. ExecFunc(MI_AI_Open(AiDevId, &stAiSetAttr), MI_SUCCESS);
```

The detailed example is as follows:

```
1. MI_AI_Attr_t stAiSetAttr = {0};
2. MI_AI_Attr_t stAiGetAttr = {0};
3. MI_AUDIO_DEV AiDevId = 0;
4. MI_AI_If_e enAiIf[] = {E_MI_AI_IF_ADC_AB};
5. MI_AI_Data_t stAiChFrame;
6. MI_AI_Data_t stAecFrame;
7. MI_U8 u8ChnGrpId = 0;
8. MI_SYS_ChPort_t stAiChnOutputPort;
9. MI_S8 s8DpgaGain[] = {-10};
10.
11. // Set the Format of AI Device to S16_LE
12. stAiSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
13.
14. // Set the Sound Mode to Mono, one Channel Group corresponds to one physical channel
15. stAiSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
16.
17. // Set the sample rate of AI Device to 8KHz
18. stAiSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
19.
20. // Set each AI Buffer to contain 1024 sampling samples
21. stAiSetAttr.u32PeriodSize = 1024;
22.
23. // Set the data arrangement form of AI buffer
24. stAiSetAttr.bInterleaved = TRUE;
25.
26. // Open AI Device
27. ExecFunc(MI_AI_Open(AiDevId, &stAiSetAttr), MI_SUCCESS);
28.
29. // Get device attributes
30. ExecFunc(MI_AI_GetAttr(AiDevId, &stAiGetAttr), MI_SUCCESS);
31.
32. // Attach ADC0/1 to WDMA1
33. ExecFunc(MI_AI_AttachIf(AiDevId, enAiIf, sizeof(enAiIf) / sizeof(enAiIf[0])), MI_SUCCESS);
34.
35. // Set output depth
36. memset(&stAiChnOutputPort, 0, sizeof(stAiChnOutputPort));
37. stAiChnOutputPort.eModId = E_MI_MODULE_ID_AI;
38. stAiChnOutputPort.u32DevId = AiDevId;
39. stAiChnOutputPort.u32ChnId = u8ChnGrpId;
40. stAiChnOutputPort.u32PortId = 0;
41. ExecFunc(MI_SYS_SetChnOutputPortDepth(0, &stAiChnOutputPort, 4, 8), MI_SUCCESS);
```

```

42.
43. // Set Interface gain of ADC0/1
44. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_AB, 18, 0), MI_SUCCESS);
45.
46. // Set DPGA gain of ADC0/1
47. ExecFunc(MI_AI_SetGain(AiDevId, u8ChnGrpId, s8DpgaGain, sizeof(s8DpgaGain) / sizeof(s8DpgaGain[0])), MI_SUCCESS);
48.
49.
50. // Enable Channel Group
51. ExecFunc(MI_AI_EnableChnGroup(AiDevId, u8ChnGrpId), MI_SUCCESS);
52.
53. // Get audio frame data
54. MI_AI_Read(AiDevId, u8ChnGrpId, &stAiChFrame, &stAecFrame, -1);
55.
56. // do something
57.
58. // Release audio frame data
59. MI_AI_ReleaseData(AiDevId, u8ChnGrpId, &stAiChFrame, &stAecFrame);
60.
61. // Disable Channel Group
62. ExecFunc(MI_AI_DisableChnGroup(AiDevId, u8ChnGrpId), MI_SUCCESS);
63.
64. // Close AI Device
65. ExecFunc(MI_AI_Close(MI_AI_DEV_1), MI_SUCCESS);

```

2.2. MI_AI_OpenWithCfgFile

➤ Features

Enable the audio input device and initialize it according to the config file.

➤ Syntax

```
MI_S32 MI_AI_GetPubAttr(MI\_AUDIO\_DEVMMI\_AUDIO\_Dev AiDevId,
MI\_AUDIO\_Attr\_t*pstAttr);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
pCfgPath	Path of config file	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: `mi_ai.h`
- Library: `libmi_ai.a/libmi_ai.so`

➤ Note

- This interface is equivalent to the combination of [MI_AI_Open](#) and [MI_AI_AttachIf](#).

- If the audio input device is enabled, it returns success.

- The config file template is as follows:

```

1. ; Device attr section
2. [DEV]
3. ; enFormat determines the data format of the sample, only supports S16_LE now.
4. ; 0[S16_LE]
5. enFormat = 0
6. ; enSoundMode determines the channel count of Channel Group.
7. ; 1[Mono] 2[Stereo] 4[4Chn] 6[6Chn] 8[8Chn]
8. enSoundMode = 2
9. ; enSampleRate determines the sample rate of AI Device.
10. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
11. enSampleRate = 8000
12. ; u32PeriodSize determines the count of sample in a AI Buffer
13. u32PeriodSize = 1024
14. ; bInterleaved determines the data arrangement mode of each channel.
15. ; if bInterleaved = 1, Data on each channel is interleaved, Otherwise, data on each channel
    is stored separately.
16. bInterleaved = 0
17. ; attach interface
18. ; 1[ADC_AB] 2[ADC_CD] 3[DMIC_A_01] 4[DMIC_A_23]
19. ; 5[I2S_A_01] 6[I2S_A_23] 7[I2S_A_45] 8[I2S_A_67] 9[I2S_A_89] 10[I2S_A_ab] 11[I2S_A_cd] 12[I
    2S_A_ef]
20. ; 13[I2S_B_01] 14[I2S_B_23] 15[I2S_B_45] 16[I2S_B_67] 17[I2S_B_89] 18[I2S_B_ab] 19[I2S_B_cd]
    20[I2S_B_ef]
21. ; 21[I2S_C_01] 22[I2S_C_23] 23[I2S_C_45] 24[I2S_C_67] 25[I2S_C_89] 26[I2S_C_ab] 27[I2S_C_cd]
    28[I2S_C_ef]
22. ; 29[I2S_D_01] 30[I2S_D_23] 31[I2S_D_45] 32[I2S_D_67] 33[I2S_D_89] 34[I2S_D_ab] 35[I2S_D_cd]
    36[I2S_D_ef]
23. ; 37[ECHO_A] 38[HDMI_A]
24. aenAiIifs={1,5}
25.
26. ; I2S RX attr section
27. ; It is not necessarywhen you're not using I2S RX
28. [I2S_A]
29. ; enMode determines the working mode of I2S Rx
30. ; 0[I2S Master] 1[I2S Slave] 2[Tdm Master] 3[Tdm Slave]
31. enMode = 0
32. ; enBitWidth determines the bit with of I2S Rx
33. ; 0[16 bit] 1[32bit]
34. enBitWidth = 0
35. ; enFormat determines the waveform alignment of I2S Rx
36. ; 0[I2S Philips] 1[I2S Left-justify]
37. enFormat = 0
38. ; enSampleRate determines the sample rate of I2S Rx
39. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
40. enSampleRate = 8000
41. ; enMclk determines the frequency of Mclk
42. ; 0[disable Mclk] 1[12.288M] 2[16.384M] 3[18.432M] 4[24.576M] 5[24M] 6[48M]
43. enMclk = 0
44. ; bSyncClock: 0[4-wire mode] 1[6-wire mode]
45. bSyncClock = 0
46. ; u32TdmSlots determines the slot number of I2S Rx
47. u32TdmSlots = 2

```

The value of the config item is consistent with the parameters given to the MI API. enFormat = 0 means AI Device uses S16_LE format, enSoundMode = 1 means Sound Mode uses Mono, enSampleRate = 8000 means AI Device uses 8KHz sample rate, u32PeriodSize = 1024 means one AI Buffer contains 1024 sample points, bInterleaved = 1 means that the channel group data arrangement is arranged in an interlaced

manner, and `aenAiIfs={1}` means that ADC0/1 is attached to WDMA. If you need to use I2S RX, you also need to set the parameters of I2S RX. The above configuration works on I2S_A, `enMode = 0` means using I2S Master mode, `enBitWidth = 0` means I2S receiving bit width is 16bit, `enFormat = 0` means aligning according to I2S Philips mode, `enSampleRate = 8000` means that the sample rate of I2S RX is 8KHz, `enMclk = 0` means that Mclk is not used, `bSyncClock = 0` means 4-wire mode, and `u32TdmSlots` means receiving 2-channel data. Please note that you need to delete the comment line when using it.



AI template.ini

➤ Example

```
1. char *path = "/tmp/Dev0Cfg.ini";
2. ExecFunc(MI_AI_OpenWithCfgFile(AiDevId, path), MI_SUCCESS);
```

2.3. MI_AI_Close

➤ Features

Close AI devices.

➤ Syntax

MI_S32 [MI_AI_Close](#) ([MI_AUDIO_DEVM](#) [MI_AUDIO_Dev](#) AiDevId);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: `mi_ai.h`
- Library: `libmi_ai.a/libmi_ai.so`

➤ Note

- If the AI device is already closed, it will return Success directly.

➤ Example

The simple example is as follows:

```
1. ExecFunc(MI_AI_Close(MI_AI_DEV_1), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.4. MI_AI_AttachIf

➤ Features

Mount peripherals to AI device.

➤ Syntax

```
MI_S32 MI_AI_AttachIf (MI_AUDIO_DEV AiDevId, const MI_AI_If_e aenAiIfs[], MI_U8 u8AiIfSize);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
aenAiIfs	AI Interface array, peripheral information that needs to be mounted to the AI device; An element of the array represents the peripherals mounted on two adjacent physical channels	Input
U8AiIfSize	Size of the interface array, the maximum is (MI_AI_MAX_CHN_NUM/2)	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- This interface can only be called after [MI_AI_Open](#) succeeds.
- If you need to mount the I2S RX to the AI device, please call [MI_AI_SetI2SConfig](#) first to initialize the I2S RX.
- If you want to get echo reference data, the AI module needs to attach E_MI_AI_IF_ECHO_A, and the AO module needs to attach E_MI_AO_IF_ECHO_A. In the AI module, E_MI_AI_IF_ECHO_A must be attached with an interface other than E_MI_AI_IF_ECHO_A, and must be placed after the interface other than E_MI_AI_IF_ECHO_A.
- When the number of attached interface channels(minus the number of channels of E_MI_AI_IF_ECHO_A) is less than or not divisible by the audio sound mode([MI_AUDIO_SoundMode_e](#)), an error is returned.

➤ Example

The simple example is as follow:

```
1. MI_AI_If_e enAiIf[] = {E_MI_AI_IF_ADC_AB, E_MI_AI_IF_ADC_CD};
2. ExecFunc(MI_AI_AttachIf(AiDevId, enAiIf, sizeof(enAiIf) / sizeof(enAiIf[0])), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.5. MI_AI_EnableChnGroup

➤ Features

Enable AI channel group.

➤ Syntax

MI_S32 [MI_AI_EnableChnGroup](#) ([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
AiChn	AI channel group number. Total audio input channels group = (the physical channels of the AI device attached to the peripheral – the physical channels occupied by the echo) / the sound mode of the AI device AI channel group number range: [0, total audio input channel group-1]	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: `mi_ai.h`
- Library: `libmi_ai.a/libmi_ai.so`

➤ Note

- This interface can only be called after [MI_AI_Open](#) and [MI_AI_AttachIf](#) succeed.
- If the AI channel group has been enabled, it will return success directly.
- The operation of the AI channel group does not support multiple processes. If a process is enabled, it can only be used and disabled here.

➤ Example

The simple example is as follow:

```
1. ExecFunc(MI_AI_EnableChnGroup(AiDevId, 0), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.6. MI_AI_DisableChnGroup

➤ Features

Disable AI channel group.

➤ Syntax

MI_S32 [MI_AI_DisableChnGroup](#) ([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- If the AI channel group has been disabled, it will return success directly.
- The operation of the AI channel group does not support multiple processes. If a process is enabled, it can only be used and disabled here.

➤ Example

The simple example is as follow:

```
1. ExecFunc(MI_AI_DisableChnGroup(AiDevId, 0), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.7. MI_AI_Read

➤ Features

Read audio data.

➤ Syntax

MI_S32 [MI_AI_Read](#) ([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx, [MI_AI_Data_t](#) *pstData, [MI_AI_Data_e](#) *pstEchoRefData, MI_S32 s32TimeoutMs);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1]	Input

Parameter Name	Description	Input/Output
	Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	
pstData	Audio data structure pointer	Output
pstEchoRefData	Echo reference data structure pointer	Output
s32TimeoutMs	Timeout for getting data: -1: blocking mode, waiting for no data; 0 means non-blocking mode, when there is no data, it will return an error; >0: blocking s32TimeoutMs milliseconds, and it will report an error and return when it times out.	Input

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - This interface can only be called after [MI_AI_Open](#) / [MI_AI_AttachIf](#) / [MI_AI_EnableChnGroup](#) succeed.
 - If you need to obtain an echo reference data, pstEchoRefData cannot be a null pointer. If you do not want to get the echo reference data pstEchoRefData, you can set it to a null pointer.
 - s32TimeoutMs value must be greater than equal to -1, -1 is equal to the data acquired using the blocking mode, the data acquired is equal to non-blocking mode 0, is greater than 0, the blocking s32TimeoutMs milliseconds, and no data timeout then return error.
 - This interface supports select operations. It is recommended to use the select/poll operation instead of timeout parameter.
- Example

The simple example is as follow:

```

1. MI_AI_Data_t stAiChFrame;
2. MI_AI_Data_t stAecFrame;
3. MI_AI_Read(AiDevId, AiChnGroup, &stAiChFrame, &stAecFrame, -1);

```

Please refer to [MI_AI_Open](#) for the detailed example.

2.8. MI_AI_ReleaseData

➤ Features

Release audio data.

➤ Syntax

```
MI_S32 MI_AI_ReleaseData (MI_AUDIO_DEV MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx,
MI_AI_Data_t MI_AI_Data_t *pstData, MI_AI_Data_t MI_AI_Data_t *pstEchoRefData);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input
pstData	Audio data structure pointer	Input
pstEchoRefData	Echo reference data structure pointer	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

None.

➤ Example

The simple example is as follow:

```
1. MI_AI_ReleaseData(AiDevId, AiChnGroup, &stAiChFrame, &stAecFrame);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.9. MI_AI_SetGain

➤ Features

Set the volume of the AI channel group.

➤ Syntax

```
MI_S32 MI_AI_SetGain (MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx, const MI_S8
as8Gains[], MI_U8 u8GainSize);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number.	Input

Parameter Name	Description	Input/Output
	Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	
as8Gains	Volume array An element of the array represents the volume of a physical channel Range: [-60, 30]	Input
u8GainSize	Size of the volume array, the maximum is the physical channel corresponding to Sound Mode	Input

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - To set the volume of the AI channel group, you must first enable the AI device and mount the peripheral.
 - Only the channels corresponding to E_MI_AI_IF_ADC_AB support this setting for Muffin series chips.
- Example

The simple example is as follow:

```
1. MI_S8 s8DpgaGain[] = {-10};
2. ExecFunc(MI_AI_SetGain(AiDevId, u8ChnGroupIdx, s8DpgaGain, sizeof(s8DpgaGain) / sizeof(s8DpgaGain[0])), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.10. MI_AI_GetGain

- Features

Get the volume of the AI channel group.
- Syntax


```
MI_S32 MI_AI_GetGain (MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx, MI_S8 as8Gains[], MI_U8 *pu8GainSize);
```
- Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input
as8Gains	Volume array An element of the array represents the volume of a physical channel Range: [-60, 30]	Output
pu8GainSize	Size of the volume array, the maximum is the physical channel corresponding to Sound Mode	Output

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - To get the volume of the AI channel group, you must first enable the AI device and mount the peripheral.
 - Only the channels corresponding to E_MI_AI_IF_ADC_AB support this setting for Muffin series chips.

➤ Example

The simple example is as follow:

```
1. MI_S8 s8DpgaGain[MI_AI_MAX_CHN_NUM];
2. MI_U8 u8Size;
3. memset(&s8DpgaGain, 0x0, sizeof(s8DpgaGain));
4. ExecFunc(MI_AI_GetGain(AiDevId, u8ChnGroupIdx, s8DpgaGain, &u8Size), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.11. MI_AI_SetMute

➤ Features

Set the mute parameters of the AI channel group.

➤ Syntax

```
MI_S32 MI_AI_SetMute (MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx, const MI_BOOL
abMutes[], MI_U8 u8MuteSize);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	Audio device ID	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input
abMutes	Mute parameter array An element of the mute parameter array represents the mute parameter of a physical channel	Input
u8MuteSize	Size of the mute parameter array, the maximum is the physical channel corresponding to Sound Mode	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

To set the mute parameters of the AI channel group, you must first enable the AI device and mount the peripheral.

➤ Example

```
1. MI_BOOL bDpgaMute[] = {TRUE, FALSE};
2. ExecFunc(MI_AI_SetMute(AiDevId, u8ChnGroupIdx, bDpgaMute, sizeof(bDpgaMute) / sizeof(bDpgaMute[0])), MI_SUCCESS);
```

2.12. MI_AI_GetMute

➤ Features

Get the mute parameters of the AI channel group.

➤ Syntax

MI_S32 [MI_AI_GetMute](#) ([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx, MI_BOOL abMutes[], MI_U8 *pu8MuteSize);

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input

Parameter Name	Description	Input/Output
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input
abMutes	Mute parameter array An element of the mute parameter array represents the mute parameter of a physical channel	Input
pu8MuteSize	Size of the mute parameter array, the maximum is the physical channel corresponding to Sound Mode	Output

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - To get the mute parameters of the AI channel group, you must first enable the AI device and mount the peripheral.
- Example


```
1. MI_BOOL bDpgaMute[MI_AI_MAX_CHN_NUM];
2. MI_U8 u8Size;
3. memset(&bDpgaMute, 0x0, sizeof(bDpgaMute));
4. ExecFunc(MI_AI_GetMute(AiDevId, u8ChnGroupIdx, s8DpgaGain, &u8Size), MI_SUCCESS);
```

2.13. MI_AI_SetIfGain

- Features

Set AI peripheral volume.
- Syntax

MI_S32 [MI_AI_SetIfGain](#) ([MI_AI_If_e](#) [MI_AI_If_e](#) enAiIf, MI_S8 s8LeftIfGain, MI_S8 s8RightIfGain);
- Parameter

Parameter Name	Description	Input/Output
enAiIf	AI peripherals	Input
s8LeftIfGain	Left channel volume	Input
s8RightIfGain	Right channel volume	Input

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - Only E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD/E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/E_MI_AI_IF_DMIC_A_45 supports this setting.
 - E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD Support volume range: Muffin/Mochi[0, 19], corresponding to 0~57Db, 3dB/step; Maruko[0, 21], corresponding to -6~57Db, 3dB/step.
 - The volume range supported by E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/E_MI_AI_IF_DMIC_A_45 is [0,6], corresponding to 0~36dB, 6dB/step.
- Example

The simple example is as follow:

```
1. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_AB, 18, 18), MI_SUCCESS);
2. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_CD, 18, 18), MI_SUCCESS);
3. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_DMIC_A_01, 4, 4), MI_SUCCESS);
4. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_DMIC_A_23, 4, 4), MI_SUCCESS);
```

Please refer to [MI_AI_Open](#) for the detailed example.

2.14. MI_AI_GetIfGain

- Features

Get AI peripheral volume.
- Syntax


```
MI_S32 MI_AI_GetIfGain (MI_AI_If_e enAiIf, MI_S8 *ps8LeftIfGain, MI_S8 *ps8RightIfGain);
```
- Parameters

Parameter Name	Description	Input/Output
enAiIf	AI peripherals	Input
ps8LeftIfGain	Left channel volume pointer	Output
ps8RightIfGain	Right channel volume pointer	Output
- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details

- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - Only
E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD/E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/E_MI_AI_IF_DMIC_A_45 supports this setting.
- Example


```
1. MI_S8 s8IfLeftGain, s8IfRightGain;
2. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_ADC_AB, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
3. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_ADC_CD, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
4. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_DMIC_A_01, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
5. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_DMIC_A_23, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
```

2.15. MI_AI_SetIfMute

- Features

Set AI peripheral mute parameters.
- Syntax

MI_S32 [MI_AI_SetIfMute](#) ([MI_AI_If_e](#) enAiIf, MI_BOOL bLeftMute, MI_BOOL bRightMute);
- Parameters

Parameter Name	Description	Input/Output
enAiIf	AI peripherals	Input
bLeftMute	Left channel mute parameter	Input
bRightMute	Right channel mute parameter	Input
- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: mi_ai.h
 - Library: libmi_ai.a/libmi_ai.so
- Note
 - No input peripheral supports this setting.
- Example


```
1. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_ADC_AB, TRUE, FALSE), MI_SUCCESS);
2. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_ADC_CD, TRUE, FALSE), MI_SUCCESS);
3. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_DMIC_A_01, TRUE, FALSE), MI_SUCCESS);
4. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_DMIC_A_23, TRUE, FALSE), MI_SUCCESS);
```


2.16. MI_AI_GetIfMute

➤ Features

Get AI peripheral mute parameters.

➤ Syntax

```
MI_S32 MI_AI_GetIfMute (MI_AI_If_e enAiIf, MI_BOOL *pbLeftMute, MI_BOOL *pbRightMute);
```

➤ Parameters

Parameter Name	Description	Input/Output
enAiIf	AI peripherals	Input
pbLeftMute	Left channel mute parameter pointer	Output
pbRightMute	Right channel mute parameter pointer	Output

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- None.

➤ Example

```
1. MI_BOOL bIfLeftMute, bIfRightMute;
2. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_ADC_AB, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
3. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_ADC_CD, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
4. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_DMIC_A_01, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
5. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_DMIC_A_23, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
```

2.17. MI_AI_SetI2SConfig

➤ Features

Set I2S RX config info.

➤ Syntax

```
MI_S32 MI_AI_SetI2SConfig (MI_AI_If_e enAiI2SIf, const MI_AUDIO_I2sConfig_t *pstConfig);
```

➤ Parameters

Parameter Name	Description	Input/Output
enAiI2Sif	Audio I2S RX input peripheral	Input
pstConfig	Audio I2S RX config information	Input

- Return value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Dependency
 - Header: `mi_ai.h`
 - Library: `libmi_ai.a/libmi_ai.so`
- Note
 - A group of I2S RX only needs to be configured once. For example, after using `E_MI_AI_IF_I2S_A_01`, I2S_A has been initialized, and then using other interfaces belonging to I2S_A, there is no need to configure I2S_A, unless the configuration parameters of I2S_A are changed.
 - The `enSampleRate` in [MI_AUDIO_I2sConfig_t](#) must be the same as the `enSampleRate` in [MI_AI_Attr_t](#), otherwise attach will report an error.

➤ Example

```

1. MI_AUDIO_I2sConfig_t stAiI2sACfg;
2. memset(&stAiI2sACfg, 0x0, sizeof(stAiI2sACfg));
3. stAiI2sACfg.enMode = E_MI_AUDIO_I2S_MODE_I2S_MASTER;
4. stAiI2sACfg.enFormat = E_MI_AUDIO_I2S_FMT_I2S_MSB;
5. stAiI2sACfg.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAiI2sACfg.enMclk = E_MI_AUDIO_I2S_MCLK_0;
7. stAiI2sACfg.bSyncClock = TRUE;
8. stAiI2sACfg.u32TdmSlots = 2;
9. stAiI2sACfg.enBitWidth = E_MI_AUDIO_BIT_WIDTH_16;
10. ExecFunc(MI_AI_SetI2SConfig(E_MI_AI_IF_I2S_A_01, &stAiI2sACfg), MI_SUCCESS);

```

2.18. MI_AI_GetI2SConfig

➤ Features

Get I2S RX config info.

➤ Syntax

`MI_S32 MI_AI_GetI2SConfig (MI_AI_If_e enAiI2Sif, MI_AUDIO_I2sConfig_t *pstConfig);`

➤ Parameters

Parameter Name	Description	Input/Output
enAiI2Sif	Audio I2S RX input peripheral	Input
pstConfig	Audio I2S RX config information	Output

- Return value
 - Zero: Successful

- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

None.

➤ Example

```
1. MI_AUDIO_I2sConfig_t stAiI2sACfg;
2. memset(&stAiI2sACfg, 0x0, sizeof(stAiI2sACfg));
3. ExecFunc(MI_AI_GetI2SConfig(E_MI_AI_IF_I2S_A_01, &stAiI2sACfg), MI_SUCCESS);
```

2.19. MI_AI_DupChnGroup

➤ Features

Sync status of AI channel group.

➤ Syntax

MI_S32 [MI_AI_DupChnGroup](#) ([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx)

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	AI device number	Input
u8ChnGrpIdx	AI channel group number. Range: [0, Channel Group Number-1] Channel Group Number = (the number of physical channels on attach-the number of Echo channels on attach) / Sound Mode	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- This is only used for dual os. It is used to synchronize the state of the AI channel group when the initialized AI channel group under RTOS is switched to Linux.

➤ Example

None.

2.20. MI_AI_InitDev

➤ Features

Initialize the AI device.

➤ Syntax

MI_S32 [MI_AI_InitDev](#) ([MI_AI_InitParam_t](#) *pstInitParam);

➤ Parameters

Parameter Name	Description	Input/Output
pstInitParam	Device initialization parameters	Input

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- It must be used in pairs with [MI_AI_DeInitDev](#), and cannot be called repeatedly, otherwise it returns to fail.
- Only used to reinitialize the AI module after the STR state is enabled.

➤ Example

None.

2.21. MI_AI_DeInitDev

➤ Features

De-Initialize the AI device.

➤ Syntax

MI_S32 [MI_AI_DeInitDev](#) (void);

➤ Parameters

Parameter Name	Description	Input/Output
----------------	-------------	--------------

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.a/libmi_ai.so

➤ Note

- This function must be called after the device is initialized, otherwise, it returns to fail.
- If it is not called before the app exits, the device will be automatically deinitialized internally
- It must be used in pairs with [MI_AI_InitDev](#), and cannot be called repeatedly, otherwise it returns to fail.

➤ Example

None.

2.22. MI_AI_GetAttr

➤ Features

Get audio input device attributes.

➤ Syntax

```
MI_S32 MI_AI_GetAttr(MI\_AUDIO\_DEV AiDevId, MI\_AI\_Attr\_t *pstAttr);
```

➤ Parameters

Parameter Name	Description	Input/Output
AiDevId	Audio input device ID	Input
pstAttr	Audio input device attributes	Output

➤ Return value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Dependency

- Header: mi_ai.h
- Library: libmi_ai.so/libmi_ai.a

※ Note

- This function must be called after opening the device successfully, otherwise it will return failed.

➤ Example

None.

3. AI DATA TYPE

The AI module related data types are defined as follows:

Data type	Definition
MI_AUDIO_DEV	Define the audio input/output device number
MI_AUDIO_Format_e	Define audio data format
MI_AUDIO_SoundMode_e	Define audio sound mode
MI_AUDIO_SampleRate_e	Define audio sample rate
MI_AI_Attr_t	Define AI device attribute structure
MI_AI_If_e	Define AI peripherals
MI_AI_MAX_CHN_NUM	Define the maximum number of physical channels supported by AI devices
MI_AI_Data_t	Define audio structure
MI_AUDIO_I2sMode_e	Define the working mode of audio I2S RX/TX
MI_AUDIO_I2sBitWidth_e	Define the bit width of audio I2S RX/TX
MI_AUDIO_I2sFormat_e	Define the data transmission format of audio I2S RX/TX
MI_AUDIO_I2sMclk_e	Define the mclk frequency of audio I2S RX/TX
MI_AUDIO_I2sConfig_t	Define I2S RX/TX config info
MI_AI_InitParam_t	AI device initialization parameters

3.1. MI_AUDIO_DEV

- Description
Define the audio input/output device number.
- Definition
typedef MI_S32 [MI_AUDIO_DEV](#)
- Note
None.
- Related data types and interfaces

None.

3.2. MI_AUDIO_Format_e

➤ Description

Define audio data format.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_FORMAT_INVALID = -1,
    E_MI_AUDIO_FORMAT_PCM_S16_LE = 0,
} MI\_AUDIO\_Format\_e;
```

➤ Member

Member Name	Description
E_MI_AUDIO_FORMAT_INVALID	Illegal data format
E_MI_AUDIO_FORMAT_PCM_S16_LE	PCM Linear 16bit (Little Endian)

➤ Note

None.

➤ Related data types and interfaces

[MI_AI_Attr_t](#)

3.3. MI_AUDIO_SoundMode_e

➤ Description

Define audio sound mode.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_SOUND_MODE_MONO = 1,
    E_MI_AUDIO_SOUND_MODE_STEREO = 2,
    E_MI_AUDIO_SOUND_MODE_4CH = 4,
    E_MI_AUDIO_SOUND_MODE_6CH = 6,
    E_MI_AUDIO_SOUND_MODE_8CH = 8,
} MI\_AUDIO\_SoundMode\_e
```

➤ Member

Member Name	Description
E_MI_AUDIO_SOUND_MODE_MONO	MONO
E_MI_AUDIO_SOUND_MODE_STEREO	STEREO
E_MI_AUDIO_SOUND_MODE_4CH	4-channels
E_MI_AUDIO_SOUND_MODE_6CH	6-channels
E_MI_AUDIO_SOUND_MODE_8CH	8-channels

➤ Note

None.

➤ Related data types and interfaces

[MI_AI_Attr_t](#)

3.4. MI_AUDIO_SampleRate_e

➤ Description

Define audio sample rate.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_SAMPLE_RATE_8000 = 8000,
    E_MI_AUDIO_SAMPLE_RATE_11025 = 11025,
    E_MI_AUDIO_SAMPLE_RATE_12000 = 12000,
    E_MI_AUDIO_SAMPLE_RATE_16000 = 16000,
    E_MI_AUDIO_SAMPLE_RATE_22050 = 22050,
    E_MI_AUDIO_SAMPLE_RATE_24000 = 24000,
    E_MI_AUDIO_SAMPLE_RATE_32000 = 32000,
    E_MI_AUDIO_SAMPLE_RATE_44100 = 44100,
    E_MI_AUDIO_SAMPLE_RATE_48000 = 48000,
    E_MI_AUDIO_SAMPLE_RATE_96000 = 96000,
    E_MI_AUDIO_SAMPLE_RATE_192000 = 192000,
}MI_AUDIO_SampleRate_e;
```

➤ Member

Member Name	Description
E_MI_AUDIO_SAMPLE_RATE_8000	8kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_11025	11.025kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_12000	12kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_16000	16kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_22050	22.05kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_24000	24kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_32000	32kHz sample rate

Member Name	Description
E_MI_AUDIO_SAMPLE_RATE_44100	44.1kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_48000	48kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_96000	96kHz sample rate
E_MI_AUDIO_SAMPLE_RATE_192000	192kHz sample rate

- Note
 - Except I2S RX which supports 8/16/32/48/96/192kHz, and I2S RX can only be attached separately when using 96/192kHz, other AI peripherals only support 8/16/32/48kHz.
 - Muffin's I2S RX only supports 8/16/32/48/96 kHz.
- Related data types and interfaces
[MI_AI_Attr_t](#)

3.5. MI_AI_Attr_t

- Description
Define AI device attribute structure.

- Definition


```
typedef struct MI_AUDIO_Attr_s
{
    MI\_AUDIO\_Format\_e enFormat;
    MI\_AUDIO\_SoundMode\_e enSoundMode;
    MI\_AUDIO\_SampleRate\_e enSampleRate;
    MI_U32 u32PeriodSize;
    MI_BOOL bInterleaved;
} MI\_AI\_Attr\_t;
```

- Member

Member Name	Description
enFormat	Audio data format. Static attributes.
enSoundMode	Audio sound mode. Static attributes. The sound mode determines how many physical channels a channel group corresponds to.
enSampleRate	Audio sampling rate. Static attributes.
u32PeriodSize	Audio data samples. Static attributes.
bInterleaved	Whether the audio data is in interlace mode. Static attributes.
E_MI_AUDIO_SAMPLE_RATE_192000	192kHz sample rate

- Note

None.

- Related data types and interfaces

[MI_AI_Open](#)

3.6. MI_AI_If_e

- Description

Define AI peripherals.

- Definition

```
typedef enum
{
    E_MI_AI_IF_NONE = 0,
    E_MI_AI_IF_ADC_AB = 1,
    E_MI_AI_IF_ADC_CD = 2,
    E_MI_AI_IF_DMIC_A_01 = 3,
    E_MI_AI_IF_DMIC_A_23 = 4,
    E_MI_AI_IF_I2S_A_01 = 5,
    E_MI_AI_IF_I2S_A_23 = 6,
    E_MI_AI_IF_I2S_A_45 = 7,
    E_MI_AI_IF_I2S_A_67 = 8,
    E_MI_AI_IF_I2S_A_89 = 9,
    E_MI_AI_IF_I2S_A_ab = 10,
    E_MI_AI_IF_I2S_A_cd = 11,
    E_MI_AI_IF_I2S_A_ef = 12,
    E_MI_AI_IF_I2S_B_01 = 13,
    E_MI_AI_IF_I2S_B_23 = 14,
    E_MI_AI_IF_I2S_B_45 = 15,
    E_MI_AI_IF_I2S_B_67 = 16,
    E_MI_AI_IF_I2S_B_89 = 17,
    E_MI_AI_IF_I2S_B_ab = 18,
    E_MI_AI_IF_I2S_B_cd = 19,
    E_MI_AI_IF_I2S_B_ef = 20,
    E_MI_AI_IF_I2S_C_01 = 21,
    E_MI_AI_IF_I2S_C_23 = 22,
    E_MI_AI_IF_I2S_C_45 = 23,
    E_MI_AI_IF_I2S_C_67 = 24,
    E_MI_AI_IF_I2S_C_89 = 25,
    E_MI_AI_IF_I2S_C_ab = 26,
    E_MI_AI_IF_I2S_C_cd = 27,
    E_MI_AI_IF_I2S_C_ef = 28,
    E_MI_AI_IF_I2S_D_01 = 29,
    E_MI_AI_IF_I2S_D_23 = 30,
    E_MI_AI_IF_I2S_D_45 = 31,
    E_MI_AI_IF_I2S_D_67 = 32,
    E_MI_AI_IF_I2S_D_89 = 33,
    E_MI_AI_IF_I2S_D_ab = 34,
    E_MI_AI_IF_I2S_D_cd = 35,
    E_MI_AI_IF_I2S_D_ef = 36,
    E_MI_AI_IF_ECHO_A = 37,
    E_MI_AI_IF_HDMI_A = 38,
```

```
E_MI_AI_IF_DMIC_A_45 = 39,  
E_MI_AI_IF_MAX,  
} MI_AI_If_e;
```

➤ Member

Member Name	Description
E_MI_AI_IF_NONE	None
E_MI_AI_IF_ADC_AB	ADC0/1
E_MI_AI_IF_ADC_CD	ADC2/3
E_MI_AI_IF_DMIC_A_01	DMIC Chn0 and Chn1
E_MI_AI_IF_DMIC_A_23	DMIC Chn2 and Chn3
E_MI_AI_IF_I2S_A_01	I2S RX A Chn0 and Chn1
E_MI_AI_IF_I2S_A_23	I2S RX A Chn2 and Chn3
E_MI_AI_IF_I2S_A_45	I2S RX A Chn4 and Chn5
E_MI_AI_IF_I2S_A_67	I2S RX A Chn6 and Chn7
E_MI_AI_IF_I2S_A_89	I2S RX A Chn8 and Chn9
E_MI_AI_IF_I2S_A_ab	I2S RX A Chn10 and Chn11
E_MI_AI_IF_I2S_A_cd	I2S RX A Chn12 and Chn13
E_MI_AI_IF_I2S_A_ef	I2S RX A Chn14 and Chn15
E_MI_AI_IF_I2S_B_01	I2S RX B Chn0 and Chn1
E_MI_AI_IF_I2S_B_23	I2S RX B Chn2 and Chn3
E_MI_AI_IF_I2S_B_45	I2S RX B Chn4 and Chn5
E_MI_AI_IF_I2S_B_67	I2S RX B Chn6 and Chn7
E_MI_AI_IF_I2S_B_89	I2S RX B Chn8 and Chn9
E_MI_AI_IF_I2S_B_ab	I2S RX B Chn10 and Chn11
E_MI_AI_IF_I2S_B_cd	I2S RX B Chn12 and Chn13
E_MI_AI_IF_I2S_D_ef	I2S RX D Chn14 and Chn15
E_MI_AI_IF_ECHO_A	SRC data (Echo)
E_MI_AI_IF_HDMI_A	HDMI RX
E_MI_AI_IF_DMIC_A_45	DMIC Chn4 and Chn5

➤ Note

None.

➤ Related data types and interfaces

[MI_AI_AttachIf](#)

[MI_AI_SetIfGain](#)

[MI_AI_GetIfGain](#)

[MI_AI_SetIfMute](#)

[MI_AI_GetIfMute](#)

[MI_AI_SetI2SConfig](#)

[MI_AI_GetI2SConfig](#)

3.7. MI_AI_MAX_CHN_NUM

- Description
Define the maximum number of physical channels supported by AI devices.
- Definition
#define [MI_AI_MAX_CHN_NUM](#) 8
- Note
None.
- Related data types and interfaces
None.

3.8. MI_AI_Data_t

- Description
Define audio structure.
- Definition

```
typedef struct MI_AI_Data_s
{
    void *apvBuffer[MI\_AI\_MAX\_CHN\_NUM];
    MI_U32 u32Bytes[MI\_AI\_MAX\_CHN\_NUM];
    MI_U64 u64Pts;
    MI_U64 u64Seq;
} MI\_AI\_Data\_t;
```

- Member

Member Name	Description
apvBuffer	Audio data address. AI device enables the interleaved mode, and the data is stored in apvBuffer[0]. AI device disables the interleaving mode, and the data is arranged in apvBuffer in order.
u32Bytes	Audio data length. One-to-one correspondence with apvBuffer.
u64Pts	Audio data timestamp. Unit: μ s.
u64Seq	Audio data sequence number.

- Note
None.
- Related data types and interfaces
[MI_AI_Read](#)
[MI_AI_ReleaseData](#)

3.9. MI_AUDIO_I2sMode_e

➤ Description

Define the working mode of audio I2S RX/TX.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_I2S_MODE_I2S_MASTER,
    E_MI_AUDIO_I2S_MODE_I2S_SLAVE,
    E_MI_AUDIO_I2S_MODE_TDM_MASTER,
    E_MI_AUDIO_I2S_MODE_TDM_SLAVE,
} MI_AUDIO_I2sMode_e;
```

➤ Member

Member Name	Description
E_MI_AUDIO_I2S_MODE_I2S_MASTER	I2S master mode
E_MI_AUDIO_I2S_MODE_I2S_SLAVE	I2S slave mode
E_MI_AUDIO_I2S_MODE_TDM_MASTER	TDM master mode
E_MI_AUDIO_I2S_MODE_TDM_SLAVE	TDM slave mode

➤ Note

Determine whether to support the master/slave mode according to different chips.

➤ Related data types and interfaces

[MI_AUDIO_I2sConfig_t](#)

3.10. MI_AUDIO_I2sBitWidth_e

➤ Description

Define the bit width of audio I2S RX/TX.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_BIT_WIDTH_16,
    E_MI_AUDIO_BIT_WIDTH_32,
} MI_AUDIO_I2sBitWidth_e;
```

➤ Member

Member Name	Description
E_MI_AUDIO_BIT_WIDTH_16	I2S bit width is 16
E_MI_AUDIO_BIT_WIDTH_32	I2S bit width is 32

➤ Note

None.

➤ Related data types and interfaces

[MI_AUDIO_I2sConfig_t](#)

3.11. MI_AUDIO_I2sFormat_e

➤ Description

Define the data transmission format of audio I2S RX/TX.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_I2S_FMT_I2S_MSB,
    E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB,
} MI\_AUDIO\_I2sFormat\_e;
```

➤ Member

Member Name	Description
E_MI_AUDIO_I2S_FMT_I2S_MSB	I2S standard format; Highest priority
E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB	I2S left-justified format; Highest priority

➤ Note

None.

➤ Related data types and interfaces

[MI_AUDIO_I2sConfig_t](#)

3.12. MI_AUDIO_I2sMclk_e

➤ Description

Define the mclk frequency of audio I2S RX/TX.

➤ Definition

```
typedef enum
{
    E_MI_AUDIO_I2S_MCLK_0,
    E_MI_AUDIO_I2S_MCLK_12_288M,
    E_MI_AUDIO_I2S_MCLK_16_384M,
    E_MI_AUDIO_I2S_MCLK_18_432M,
    E_MI_AUDIO_I2S_MCLK_24_576M,
    E_MI_AUDIO_I2S_MCLK_24M,
    E_MI_AUDIO_I2S_MCLK_48M,
```

} [MI_AUDIO_I2sMclk_e](#);

➤ Member

Member Name	Description
E_MI_AUDIO_I2S_MCLK_0	Turn off MCLK
E_MI_AUDIO_I2S_MCLK_12_288M	Set MCLK to 12.88M
E_MI_AUDIO_I2S_MCLK_16_384M	Set MCLK to 16.384M
E_MI_AUDIO_I2S_MCLK_18_432M	Set MCLK to 18.432M
E_MI_AUDIO_I2S_MCLK_24_576M	Set MCLK to 24.576M
E_MI_AUDIO_I2S_MCLK_24M	Set MCLK to 24M
E_MI_AUDIO_I2S_MCLK_48M	Set MCLK to 48M

➤ Note

None.

➤ Related data types and interfaces

[MI_AUDIO_I2sConfig_t](#) [MI_AUDIO_I2sConfig_t](#)

3.13. [MI_AUDIO_I2sConfig_t](#)

➤ Description

Define I2S RX/TX config info.

➤ Definition

```
typedef struct MI_AUDIO_I2sConfig_s
{
    MI\_AUDIO\_I2sMode\_e enMode;
    MI\_AUDIO\_I2sBitWidth\_e enBitWidth;
    MI\_AUDIO\_I2sFormat\_e enFormat;
    MI\_AUDIO\_SampleRate\_e enSampleRate;
    MI\_AUDIO\_I2sMclk\_e enMclk;
    MI_BOOL bSyncClock;
    MI_U32 u32TdmSlots;
} MI\_AUDIO\_I2sConfig\_t;
```

➤ Member

Member Name	Description
enMode	I2S working mode
enBitWidth	I2S data bit width.
enFormat	I2S transmission data format.
enSampleRate	I2S sampling rate.
enMclk	I2S mclk frequency.
bSyncClock	Whether I2S RX and I2S TX share the clock.
u32TdmSlots	Number of I2S TDM slots (valid only in TDM mode)

➤ Note

None.

- Related data types and interfaces
[MI_AI_SetI2SConfig](#)
[MI_AI_GetI2SConfig](#)

3.14. MI_AI_InitParam_t

- Description
AI device initialization parameters.

- Definition

```
typedef struct MI_AI_InitParam_s
{
    MI\_AUDIO\_DEV AiDevId;
    MI_U8 *u8Data;
} MI\_AI\_InitParam\_t;
```

- Member

Member Name	Description
AiDevId	AI device number
u8Data	Initialization parameter pointer (reserved)

- Note
None.

- Related data types and interfaces
[MI_AI_InitDev](#) [MI_AI_InitDev](#)

4. ERROR CODE

The AI API error codes are shown in the table below:

Error Code	Definition	Description
0xA0042001	MI_AI_ERR_INVALID_DEVID	Invalid audio input device number
0xA0042002	MI_AI_ERR_INVALID_CHNGRPID	Invalid audio input channel group number
0xA0042003	MI_AI_ERR_ILLEGAL_PARAM	Invalid audio input parameter setting
0xA0042006	MI_AI_ERR_NULL_PTR	Input parameter empty indicator error
0xA0042007	MI_AI_ERR_NOT_CONFIG	Audio input device properties are not set
0xA0042008	MI_AI_ERR_NOT_SUPPORT	Operation is not supported
0xA0042009	MI_AI_ERR_NOT_PERM	Operation not allowed
0xA004200C	MI_AI_ERR_NOMEM	Failed to allocate memory
0xA004200D	MI_AI_ERR_NOBUF	Insufficient audio input buffer
0xA004200E	MI_AI_ERR_BUF_EMPTY	Audio input buffer is empty
0xA004200F	MI_AI_ERR_BUF_FULL	Audio input buffer is full
0xA0042010	MI_AI_ERR_SYS_NOTREADY	Audio input system is not initialized
0xA0042012	MI_AI_ERR_BUSY	Audio input system is busy
0xA0042017	MI_AI_ERR_NOT_ENABLED	Audio input device or channel is not enabled

5. PROCFS INTRODUCTION

5.1. cat

➤ Debug info

```
# cat proc/mi_modules/mi_ai/mi_ai0
```

```
===== MI AI Dev0 Info =====

----- Start AI Dev0 Attr -----
DevStatus   Format   SoundMode   SampleRate   PeriodSize   Interleaved
started     S16_LE   Stereo      8000         1024         1

DmaBufSize   DmaBusySize   DmaFreeSize   TmpBufSize
65536        1088          64448         4096

Mha1ChnCnt   MiChnGrpCnt   ReadCountFromMha1
2            1             63

If slot[0]: ADC_AB
ADC_AB      :      Gain(0 ,0 ) Mute(0,0).

----- End AI Dev0 Attr -----

----- Start AI ChnGrp0 info -----
ChnGrpId    bEnable    ReadFrameCount
0           1           63
Mute Status:
Mute[0]: 0
Mute[1]: 0

Dpga Gain:
Dpga Gain[0]: 0
Dpga Gain[1]: 0

----- End AI ChnGrp0 info -----
```

➤ Debug info analysis

Record the current AI usage status and device/channel group attributes, and can dynamically obtain information, which is convenient for debugging and testing.

➤ Parameter description

Parameter		Description
AI Device Attr	DevStatus	AI device name uninit: ununit opened: open successfully
	Format	Audio format (Bit width and size end, etc) Currently only support S16_LE (16bit, small end mode)
	SoundMode	Sound mode: mono stereo SOUND_MODE_4CH: 4channel SOUND_MODE_6CH: 6channel SOUND_MODE_8CH: 8channel
	SampleRate	8k/16k/32k/48k
	PeriodSize	AI data size per frame
	Interleaved	Whether it is interleaved mode
	DmaBufSize	DMA buffer size
	DmaBusySize	DMA buffer busy size
	DmaFreeSize	DMA buffer free size
	MhalChnCnt	Channel count of mhal device
	MiChnGrpCnt	MI channel group count
	ReadCountFromMhal	Frame count read by AI from mhal
	If slot[IfIdx]: interface type	[IfIdx]: Interface index interface type
	[interface type]:Gain(Leftgain,RightGain) Mute(LeftMute,RightMute)	[interface type] Gain(Leftgain,RightGain) Mute(LeftMute,RightMute)

Parameter		Description
AI I2S Status	I2sMode	I2S Rx workmode (Only valid when interface is I2S Rx) i2s-master i2s-slave tdm-master tdm-slave
	I2sMclk	I2S Rx Mclk frequency (Only valid when interface is I2S Rx) disable: not use Mclk Other values are the current Mclk frequency.
	I2sFmt	I2S Rx data format (Only valid when interface is I2S Rx) I2S-MSB: I2S format LEFT-MSB: I2S Left-justified format
	bI2sSync	Whether I2S RX and TX share clock(Only valid when the device is I2S Rx) 1: 4 wire mode, RX and TX share clock 0: 6 wire mode, RX and TX have independent clocks
	TdmSlots	I2S Rx TDM slot number(Only valid when the device is I2S Rx and in TDM mode)
	I2sBitWidth	I2S RX Bit Width(Only valid when the device is I2S Rx and the chip supports TDM mode)
AI ChnGrp info	ChnGrpId	MI channel group ID
	bEnable	Whether to enable
	ReadFrameCount	Frame count read by the corresponding channel group
	Mute Status	DPGA Mute info
	Mute[PhyChnIdx]:bMute	[PhyChnIdx]: channel index in channel group bMute: mute status
	Dpga Gain	DPGA gain info
	Dpga Gain[PhyChnIdx]:Gain	[PhyChnIdx]: channel index in channel group Gain

5.2. echo

Function	Dynamically enable/disable AI device to dump pcm data
Command	echo dump_mhal_chn [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter description	[Path] Path to save dumped file
	[ON/on/1, OFF/off/0] Enable or not
	[ID] AI device ID
Example	echo dump_mhal_chn /tmp 1 > proc/mi_modules/mi_ao/mi_ai0

Function	Dynamically enable/disable AI device to dump pcm data
Command	echo dump_mi_chn [ChnGrpId] [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter description	[ChnGrpId] channel group ID
	[Path] Path to save dumped file
	[ON/on/1, OFF/off/0] Enable or not
	[ID] AI device ID
Example	echo dump_mi_chn 0 /tmp 1 > proc/mi_modules/mi_ao/mi_ai0

Function	Dynamically set the gain of AI DPGA
Command	echo set_dpga_gain [ChnGrpId] [ChnId] [Gain] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter description	[ChnGrpId] channel group ID
	[ChnId] channel index in channel group
	[Gain] AI digital gain
Example	echo set_dpga_gain 0 0 -10 > proc/mi_modules/mi_ao/mi_ai0

Function	Dynamically set the gain of AI Interface
Command	echo set_inf_gain [If] [LeftGain] [RightGain] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter description	[If] AI Interface
	[LeftGain] [RightGain]
Example	echo set_inf_gain 1 18 18 > proc/mi_modules/mi_ao/mi_ai0

Function	Dynamically set AI DPGA mute mode
Command	echo set_dpga_mute [ChnGrpId] [ChnId] [Mute] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter description	[ChnGrpId] channel group ID
	[ChnID] channel index in channel group
	[Mute] AI DPGA mute
Example	echo set_dpga_mute 0 0 1 > proc/mi_modules/mi_ao/mi_ai0

Function	Dynamically set AI Interface silent mode
----------	--

Command	echo set_inf_mute [If] [LeftMute] [RightMute] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter	[If] AI Interface
description	[LeftMute] [RightMute]
Example	echo set_inf_mute 1 0 1 > proc/mi_modules/mi_ao/mi_ao0

Function	Dynamically enable/disable Singen of AI Device
Command	echo singen [SinGen Index] [Enable] > proc/mi_modules/mi_ai/mi_ai[ID]
Parameter	[SinGen Index] Singen ID
description	[Enable] enable/disable Singen
Example	echo singen 0 1 > proc/mi_modules/mi_ao/mi_ao0